

TECH CUP - Engineering Assignment

¡Bienvenidos al Sprint #2 del desafío técnico de TechCup! Esta semana continuamos con el desarrollo progresivo del proyecto. Durante este sprint se introducirán nuevas implicaciones que deben ser analizadas e integradas en la solución actual. El objetivo de este ejercicio es simular un entorno real de ingeniería, donde los sistemas evolucionan sprint tras sprint y el equipo debe adaptar su solución de acuerdo al feedback obtenido.

CONSIDERACIONES DEL TRABAJO EN EQUIPO

Para el desarrollo del proyecto deberán cumplir con las siguientes dinámicas de trabajo:

1. Dailies:

Cada equipo deberá realizar **reuniones diarias (Daily)** de aproximadamente **15 minutos**. En estas reuniones cada integrante debe responder:

- Qué hice ayer con respecto a mis asignaciones
- Qué haré hoy
- Qué bloqueos tengo

NOTA: Si no pueden realizar la reunión virtual, pueden hacerla presencial, pero deben dejar evidencia escrita que incluya:

- resumen de lo discutido por todos los integrantes
- integrantes que asistieron
- decisiones tomadas

2. Comunicación del Equipo:

El equipo debe tener un canal de comunicación activo para coordinar el proyecto, puede ser mediante:

- **Microsoft Teams**
- **WhatsApp**

El docente debe estar **incluido en este canal** para poder observar el proceso de colaboración del equipo.

3. Organización del Equipo:

Cada sprint deberán compartir:

- **Organigrama del equipo:** Debe incluir sus roles asignados, este organigrama puede cambiar de sprint a sprint.
- **Disponibilidad del equipo:** Cada integrante debe indicar: horarios disponibles y posibles restricciones de tiempo según sus actividades

académicas o laborales

Esto permitirá planear correctamente el trabajo del sprint.

ACTIVIDADES DEL SPRINT #2

Durante este sprint deberán realizar las siguientes actividades

1. IDENTIFICACIÓN Y SOLUCIÓN DE LA DEUDA TÉCNICA SPRINT #1:

Antes de continuar con el desarrollo del sistema, el equipo debe revisar el estado actual del proyecto junto con los comentarios de feedback que se presentaron en la sesión de laboratorio por parte del profesor.

Entre las actividades se incluyen:

- Revisar los requerimientos establecidos y actualizarlos con el feedback recomendado.
- Revisar el análisis de requerimientos y actualizarlo con los comentarios realizados.
- Identificación de inconsistencias en las definiciones de los artefactos entregados:
 - Mockups del sistema - Correcciones de forma y de estructura, navegación, y flujos por roles.
 - Manual de identidad visual - Correcciones de logotipos, colores utilizados y explicación técnica como soporte.
 - Repositorio Back End - Readme tecnico
 - Repositorio Front End - Generación y correcciones en el Readme tecnico
 - Diagramas del sistema - Correcciones:
 - Diagrama de Contexto
 - Diagrama de Clases
 - Diagrama de componentes General
 - Diagrama de componentes Específico
 - Detección de deuda técnica - Puntos faltantes si no fueron desarrollados
 - Funcionalidades implementadas en BackEnd
 - Set de pruebas y su estructura
 - Organización de la paquetería de manera adecuada

2. REFINAMIENTO DEL BACKLOG:

Una vez identificada la deuda técnica, el equipo deberá organizar el trabajo del proyecto dentro de JIRA:

A. Product Backlog:

- a. Dentro de cada una de las ÉPICAS establecidas, genere las tareas necesarias asociadas a la deuda técnica presentada en el Sprint #1

NOTA: Cada deuda técnica identificada debe registrarse como una feature o tarea independiente dentro del backlog atada a la épica correspondiente.

- b. Genere un feature asociado a la Implementación del Swagger dentro del proyecto, recuerde que tiene que fragmentarlo en las tareas necesarias:
 - i. Implementación de la dependencia dentro del pom.xml
 - ii. Implementación de las anotaciones dentro de los controladores (Especificar cuáles controladores en la descripción)
 - iii. Construcción de la clase de configuración para la API, para personalizar el swagger ui
 - iv. Prueba funcional en el navegador de la implementación
- c. Genere un feature asociado a la Implementación de la observabilidad con logs dentro del sistema, recuerde que tiene que fragmentarlo en las tareas necesarias:
 - i. Implementación de la dependencia dentro del pom.xml
 - ii. Creación de las configuraciones application.yaml dentro del proyecto
 - iii. Definición del nivel de logs a manejar
 - iv. Implementación de Logs dentro de las funcionalidades que ya fueron creadas e implementadas en Sprint #1
 - v. Implementación de Logs dentro de las funcionalidades para el Sprint #2
- d. Para las funcionalidades necesarias para el sistema, tanto las que ya fueron implementadas en sprint #1 como las que están en el backlog, se le sugiere que para las tareas de backend agregue las siguientes tareas de ejemplo:
 - i. Creación de Mappers - Dto a Modelo
 - ii. Creación de Mappers - Modelo a Dto
 - iii. Pruebas de los Mappers
 - iv. Pruebas de los Validadores
 - v. Pruebas de las Utilidades
 - vi. Manejo de las Excepciones:
 - 1. Personalizadas
 - 2. De negocio
 - 3. Genéricas
 - vii. Implementación del Error Handler como clase
 - viii. Implementación de las respuestas dependiendo el tipo de excepción a manejar

NOTA: Son sugerencias para que el equipo pueda iniciar a trabajar correctamente, si necesita más tareas que cumplan con la creación y ejecución correcta de dicha funcionalidad se pueden agregar.

- e. Para las feature de arquitectura incluya:
 - i. Diagramas de Secuencia - Para Todas las funcionalidades establecidas del sistema

- f. Para el Front End en la época establecida deben ir las funcionalidades visuales del sistema, para este sprint **ÚNICAMENTE** deben continuar con las sub tareas relacionadas con los MockUps.

3. REFINAMIENTO DEL SET DE PRUEBAS:

A partir de lo visto en clase con respecto a la cobertura y análisis de código deberán agregar a la definición de los escenarios de prueba para cada funcionalidad:

- Evidencias de los registros de cobertura a nivel de Líneas, Ramas, Métodos y Clases con Jacoco
- Evidencia de la ejecución con SonarQube del análisis estático de código
 - Si los calificativos son menores a B tiene que realizar las tareas correspondientes para mejorar y subir sus puntuaciones.

Estos deben agregarse a JIRA como evidencia dentro de cada feature.

4. CONSTRUCCIÓN DEL SPRINT BACKLOG SPRINT #2:

- A. Después de organizar correctamente la planeación del trabajo, Identifique cuáles tareas son más prioritarias, cuáles tareas tienen correlación directa, que tarea puede bloquear a otra o es derivada de otra.
- B. El líder de la semana anterior tiene que cerrar el Sprint #1, resumiendo en el cierre que se logró y que quedó como deuda técnica para el Sprint #2.
- C. Del Backlog Seleccione las actividades que planean realizar y desarrollar en este sprint, recuerde que tiene que ser concordante con la matriz de trazabilidad que realizaron en Sprint #1 - Adicional recuerde la cantidad de semanas de trabajo, distribuyan bien la carga laboral.
- D. Arrastre cada una de esas tareas al Sprint Backlog del Sprint #2.
- E. Una vez seleccionadas, realicen la correcta estimación de las tareas, recordando usar Planning Poker para la definición, Fibonacci para los puntajes y recordando que la carga máxima por persona. - Recuerde el balance entre todos los integrantes del equipo.
- F. Finalmente deberán Iniciar el Sprint en JIRA y definir el objetivo para este SPRINT #2.

5. CREACIÓN DE DIAGRAMAS DE ARQUITECTURA:

El equipo deberá continuar con el diseño de la arquitectura del sistema. Deben construir los siguientes diagramas y agregarlos como índices dentro del README de su backend:

- **Diagramas de Secuencia:** Enfocado en el detalle de cómo interactúan los componentes internos del sistema para una determinada funcionalidad.

6. IMPLEMENTACIÓN DE CÓDIGO:

Durante este sprint deberán continuar con el desarrollo a nivel de código de sus funcionalidades, utilizando una arquitectura MVC.

Deben implementar:

- Por funcionalidad escogida y tareas a realizar este sprint, deberán crear todo el flujo desde el controlador hasta la creación de los servicios, utilidades y validadores.
- Pruebas de acuerdo al set de pruebas que surge para esa funcionalidad que se va a trabajar.

Para este sprint **no se implementará persistencia en base de datos**. Por lo tanto deberán utilizar datos falsos o simulados dentro de los servicios para representar el comportamiento del sistema, pueden simular la persistencia guardando en listas de objetos complejos, diccionarios o lo que les sea más cómodo.

7. SPRINT REVIEW:

Al finalizar el sprint deberán realizar una presentación del trabajo realizado, la cual será expuesta en la sesión de laboratorio por cada líder técnico. La presentación debe incluir:

- **Miembros del equipo y que rol tuvo cada uno esta semana**
- **Objetivo del sprint:** Qué buscaba lograr el equipo durante esta semana.
- **Distribución de tareas:** Cómo se repartió el trabajo dentro del equipo, Como quedo la matriz de trazabilidad y el sprint backlog para el SPRINT #2.
- **Métricas del Sprint:** Puntos asignados a cada integrante, Puntos logrados por cada integrante y puntos totales del sprint
- **Demo funcional:** Demostración del sistema funcionando y ejecutado con spring boot - Correcciones realizadas y Que lograron avanzar este sprint

8. SPRINT RETROSPECTIVE: Retroalimentación del Equipo

Finalmente el equipo debe reflexionar sobre el proceso respondiendo:

- ¿Qué hicieron bien?
- ¿Qué hicieron mal?
- ¿Qué mejorarían en el siguiente sprint?
- ¿A qué se comprometen para el próximo sprint?

ENTREGABLES:

Dentro del espacio de moodle deben enviar:

- **Link de su repositorio BackEnd**
- **Link de su repositorio FrontEnd**



- **Link de sus mockups con Figma**
- **Documento de Análisis de Requerimientos**
- **Documento de Manual de Identidad**
- **Presentación de Canva o Powerpoint de la retrospectiva y review**